

```
##### 1. 填补空缺的SECOM数据，做降维
sedata = '/Users/ekeulseuji/Downloads/secom/secom.data'
selabels = '/Users/ekeulseuji/Downloads/secom/secom_labels.data'
senames = '/Users/ekeulseuji/Downloads/secom/secom.names'

df_data <- read.table(sedata, header=FALSE, sep="",
                      fill=TRUE, na.strings=c("NA"))

# 看label的内容
label_lines <- readLines(selabels, n=10)
print(label_lines)

df_labels <- read.table(selabels, header=FALSE, col.names=c("Label", "Time"),
                        stringsAsFactors=FALSE)

dim(df_data) # 1567 590
dim(df_labels) # 1567 3

df_combined <- cbind(df_data, df_labels)
head(df_combined[, 1:2])
# 1    -1 19/07/2008 11:55:00      Pass
# 2    -1 19/07/2008 12:32:00      Pass

table(df_combined$Label) # 再看label的分布 (imbalanced, 之后试SMOTE)
#    -1     1
# 1463  104

# 用每一列的平均值去补NA并检查NA情况
df_data_filled <- df_data

for(i in 1:ncol(df_data_filled)) {
  if(any(is.na(df_data_filled[, i]))) {
    col_mean <- mean(df_data_filled[, i], na.rm=TRUE)
    df_data_filled[is.na(df_data_filled[, i]), i] <- col_mean}
}
sum(is.na(df_data_filled))

# 移除列内方差为0的列 (共116列)
# 要不然cannot use 'cor = TRUE' with a constant variable
constant_vars <- sapply(df_data_filled, function(x) length(unique(x))==1)
df_data_filtered <- df_data_filled[, !constant_vars]

df_pca <- princomp(df_data_filtered, cor=T, scores=T)

summary(df_pca)
# Importance of components:
#
#          Comp.1      Comp.2      Comp.3
# Standard deviation  5.13335016 4.14492074 3.65500452
# Proportion of Variance 0.05559343 0.03624550 0.02818367

df_pca$loadings
# Comp.1 Comp.2
# SS loadings    1.000  1.000
# Proportion Var  0.002  0.002

dim(df_pca$scores)
# 1567  474

color_pca <- ifelse(df_combined$Label[1:nrow(df_data_filtered)]== -1,
                    "lightseagreen", "salmon")
plot(df_pca$scores[, 1:2], col=color_pca, xlab="PC1", ylab="PC2")
```

```

# 用标准差算出各PC对总方差的解释率和累积方差解释率
vars <- df_pca$sdev^2
explained_variance_ratio <- vars / sum(vars)
cumulative_var <- cumsum(explained_variance_ratio)

# 画累计图 参考https://plotly.com/r/pca-visualization/
library(plotly)
data <- data.frame(Explained_Variance=cumulative_var,
                   Components=1:length(cumulative_var))
plot_ly(data=data, x=~Components, y=~Explained_Variance,
        type='scatter', mode='lines', fill='tozero') %>%
layout(title="Cumulative Explained Variance",
       xaxis=list(title="Number of Components"),
       yaxis=list(title="Cumulative Explained Variance"))

# 对df_combined做一样的填补缺失值和去除无方差列
for(i in 1:ncol(df_combined)) {
  if(any(is.na(df_combined[, i]))) {
    col_mean <- mean(df_combined[, i], na.rm = TRUE)
    df_combined[is.na(df_combined[, i]), i] <- col_mean}
}
constant_vars <- sapply(df_combined, function(x) length(unique(x))==1)
constant_names <- names(df_combined)[constant_vars]
df_combined_filtered <- df_combined[, !constant_vars]

# 做scaling调整数据尺度方便特征距离计算
X <- scale(df_combined_filtered[, -ncol(df_combined_filtered)])
df_combined_filtered$Label <- factor(df_combined_filtered$Label,
                                   levels=c(-1, 1), labels=c("Pass", "Fail"))
unique(df_combined_filtered$Label)

set.seed(42)
tsne_out <- Rtsne(X, dims=2, perplexity=30, verbose=TRUE)

tsne_df <- data.frame(tSNE1=tsne_out$Y[,1], tSNE2=tsne_out$Y[,2],
                    Label=df_combined_filtered$Label)

# t-SNE
ggplot(tsne_df, aes(x=tSNE1, y=tSNE2, color=Label)) +
  geom_point(alpha=0.6) + theme_minimal() +
  labs(title=" ", color="Class") +
  scale_color_manual(values=c("Pass"="lightseagreen", "Fail"="salmon"))

# LDA, 还有训练集用SMOTE补齐label占比不平衡的LDA
library(MASS)

df_labels$Label_Factor <- as.factor(ifelse(df_labels$Label==-1, "Pass", "Fail"))
df_combined_filtered <- cbind(df_data_filtered, Label=df_labels$Label_Factor)

n <- nrow(df_combined_filtered)
index <- sample(n)
train <- df_combined_filtered[index[1:floor(0.7*n)], ]
test <- df_combined_filtered[index[(floor(0.7*n) + 1):n], ]

lda_model <- lda(Label ~ ., data=train)

pred <- predict(lda_model, test)
table(pred$class, test$Label)
#      Fail Pass
# Fail      8  29
# Pass     29 405

library(smotefamily)

```

```

X_train <- train[, -ncol(train)]
y_train <- train$Label
smote_res <- SMOTE(X_train, y_train, K=5, dup_size=2)

train_smote <- smote_res$data
colnames(train_smote)[ncol(train_smote)] <- "Label"
train_smote$Label <- as.factor(train_smote$Label)

table(train_smote$Label)

lda_model_smote <- lda(Label ~ ., data=train_smote)

# 结果算敏感和特异度是更差, Fail类的样本太少而且看tSNE图PassFail混在一起很难分开
pred_smote <- predict(lda_model_smote, test)
table(pred_smote$class, test$Label)
#      Fail Pass
# Fail   10   37
# Pass   27  397

##### 2. 对Ceramic数据集做聚类并且找出典型类
cedata = read.csv('/Users/ekulseuji/Downloads/Chemical Composition of Ceramic.csv',
  header=TRUE)

head(cedata)
colSums(is.na(cedata))

# 对数值数据做scaling调整数据尺度方便特征距离计算
numeric_cols <- sapply(cedata, is.numeric)
non_numeric_cols <- !numeric_cols
cedata_numeric <- cedata[, numeric_cols, drop = FALSE] # drop=FALSE
cedata_non_numeric <- cedata[, non_numeric_cols, drop = FALSE]

cedata_numeric_scaled <- scale(cedata_numeric)
ce_df <- as.data.frame(cedata_numeric_scaled)
cedata <- cbind(cedata_non_numeric, cedata_numeric_scaled)

# K means
library(factoextra)
fviz_nbclust(cedata_numeric_scaled, kmeans, method="wss") +
  geom_vline(xintercept=2, linetype=2)

km <- kmeans(cedata_numeric_scaled, 2, nstart=24)
print(km)

fviz_cluster(km, data=cedata_numeric_scaled, ellipse.type="euclid",
  star.plot=TRUE, repel=TRUE, ggtheme=theme_minimal())

# Gaussian Mixture Models
library(mclust)
table(cedata$Ceramic.Name)
EM <- Mclust(cedata_numeric_scaled, G=2)
summary(EM, parameters=T)
# plot(EM, what="classification")
library(factoextra)
p <- fviz_mclust(EM, what="classification")
p + theme_minimal() + labs(title="Gaussian Mixture Models")

# 降维做可视化
pca_res <- prcomp(ce_df, center=TRUE, scale.=TRUE)
pca_df <- as.data.frame(pca_res$x[, 1:2])

# library("Rtsne")

```

```

# set.seed(42)
# tsne_out <- Rtsne(as.matrix(ce_df), dims=2, pca=FALSE, perplexity=20, theta=0.0)
# tsne_df <- as.data.frame(tsne_out$Y)
# names(tsne_df) <- c("tSNE1", "tSNE2")

# 层次聚类
result <- dist(cedata_numeric_scaled, method="euclidean")
result_hc <- hclust(d=result, method="ward.D2")
# fviz_dend(result_hc, k=2, cex=0.7, lwd=0.5, rect=TRUE)
prd = cutree(result_hc, k=2)
table(prd)
library("ggplot2")
p <- fviz_dend(result_hc, k=2, cex=0.7, rect=FALSE)
p$layers[[1]]$aes_params$linewidth <- 0.5
p
hc_clusters <- cutree(result_hc, k=2)
pca_df$Cluster <- as.factor(hc_clusters)
# tsne_df$Cluster <- as.factor(hc_clusters)

ggplot(pca_df, aes(x=PC1, y=PC2, color=Cluster)) + geom_point(size=3, alpha=0.7) +
  stat_ellipse(level=0.95, linetype=2) + theme_minimal() + labs(title =
    "Hierarchical Clustering", x = paste0("PC1 (",
      round(summary(pca_res)$importance[2, 1]*100, 1), "%)" , y = paste0("PC2 (",
      round(summary(pca_res)$importance[2, 2]*100, 1), "%)" , color = "Cluster")

# 谱聚类
library("kernlab")
dim(ce_df)
# ggplot(ce_df, aes(x=Na2O, y=MgO))+geom_point()
set.seed(123)
km_result <- kmeans(ce_df, 2, nstart=24) # K-means可视化展示
fviz_cluster(km_result, ce_df, geom="point", ellipse=FALSE,
  show.clust.cent=FALSE, palette="jco", ggtheme=theme_classic())
sc <- specc(as.matrix(ce_df), centers=2) # 进行spectral clustering
# plot(as.matrix(ce_df), col=sc)

pca_df$Cluster <- as.factor(sc@.Data)
# tsne_df$Cluster <- as.factor(sc@.Data)

# df1<-ce_df # 将类标签和原始数据融合
# df1$class<-as.factor(sc@.Data)
# ggplot(tsne_df, aes(x=tSNE1, y=tSNE2, color=Cluster)) + geom_point()

ggplot(pca_df, aes(x=PC1, y=PC2, colour=Cluster)) + geom_point(size=3, alpha=0.7) +
  stat_ellipse(level=0.95) + theme_minimal() + labs(title="Spectral Clustering",
  x=paste0("PC1 (", round(summary(pca_res)$importance[2, 1]*100, 1), "%)" ,
  y=paste0("PC2 (", round(summary(pca_res)$importance[2, 2]*100, 1), "%)" ))

# 看分类结果混淆矩阵
table(cedata_non_numeric$Part) # table(sc@.Data)
cedata_non_numeric$Ceramic.Name[km$cluster==1]
cedata_non_numeric$Ceramic.Name[hc_clusters==1]

true_labels <- ifelse(cedata_non_numeric$Part=="Glaze", 1, 2)
pred_kmeans <- km$cluster
table(pred_kmeans, true_labels)
pred_gmm <- EM$classification
table(pred_gmm, true_labels)
pred_hc <- prd
table(pred_hc, true_labels)
pred_sc <- sc@.Data
table(pred_sc, true_labels)

```

```

mnist_dat = read.csv("/Users/ekeulseuji/Downloads/mnist_train.csv")

dim(mnist_dat) # 59999 785
head(mnist_dat)
head(mnist_dat[,1]) # label 0 4 1 9 2 1, [,2:785]是像素值
colnames(mnist_dat)[1] = "label"
colnames(mnist_dat)[2:785] = paste0("pixel_val",1:784)
mnist_dat$label <- as.factor(mnist_dat$label)

set.seed(210766)
n = nrow(mnist_dat)
index = sample(n)
train = mnist_dat[index[1:floor(0.8*n)], ]
test = mnist_dat[index[(floor(0.8*n)+1):n], ]

# Naive Bayes
library(e1071)
nb_mod <- naiveBayes(label~., data=train)
nb_pred <- predict(nb_mod, test)

nb_tab = table(nb_pred, test$label)
nb_acc = sum(diag(nb_tab))/sum(nb_tab)
nb_tab
nb_acc

# SVM
library(kernlab)
svm_mod <- ksvm(label~., data=train, type="C-bsvc", kernel="rbfdot")
svm_pred <- predict(svm_mod, test)

svm_tab = table(svm_pred, test$label)
svm_acc = sum(diag(svm_tab))/sum(svm_tab)
svm_tab
svm_acc

# CART
library(rpart)
cart_mod <- rpart(label~., data=train, method='class')
cart_pred <- predict(cart_mod, test, type="class")

cart_tab = table(cart_pred, test$label)
cart_acc = sum(diag(cart_tab))/sum(cart_tab)
cart_tab
cart_acc # 0.61625

# 调参参考 https://cran.r-project.org/web/packages/rpart/rpart.pdf
cart_mod <- rpart(label~., data=train, method='class',
                  control=rpart.control(minsplit=35, cp=0.001, maxdepth=30))
cart_pred <- predict(cart_mod, test, type="class")

cart_tab = table(cart_pred, test$label)
cart_acc = sum(diag(cart_tab))/sum(cart_tab)
cart_tab
cart_acc # 0.77375

# LDA

```

```

library(MASS)
# variables ...784 appear to be constant within groups
# lda_mod <- lda(label~., data=train)
# lda_pred <- predict(lda_mod, test)

const_vars_train <- sapply(train, function(x) length(unique(x)) == 1)
train_filtered <- train[, !const_vars_train]
test_filtered <- test[, names(train_filtered)]

lda_mod <- lda(label~., data=train_filtered)
lda_pred <- predict(lda_mod, test_filtered)

lda_tab = table(lda_pred$class, test_filtered$label)
lda_acc = sum(diag(lda_tab))/sum(lda_tab)
lda_tab
lda_acc

# Logistic Regression
# 参考 https://www.r-bloggers.com/2020/05/multinomial-logistic-regression-with-r/
library(nnet)
# train$label <- relevel(train$label, ref="0")
# multinom_mod <- multinom(label~., data=train, maxit=500, trace=FALSE)
# multinom_pred <- predict(multinom_mod, test)
# Error in nnet.default(X, Y, w, mask = mask, size = 0, skip = TRUE, softmax =
# TRUE, : too many (7860) weights

# 用pca做降维
pca_res <- prcomp(train[, -1], center=TRUE, scale=FALSE)
cum_var <- cumsum(pca_res$sdev^2/sum(pca_res$sdev^2))
head(cum_var)
n_comp <- which(cum_var>=0.9)[1]
n_comp # 87
train_pca <- data.frame(label=train$label, pca_res$x[, 1:n_comp])
# 把train数据里的pca降维方式套用在test数据上
test_pca <- data.frame(label=test$label, predict(pca_res, test[, -1])[, 1:n_comp])

train$label <- relevel(train_pca$label, ref="0")
multinom_mod <- multinom(label~., data=train_pca, maxit=500, trace=FALSE)
multinom_pred <- predict(multinom_mod, test_pca)

multinom_tab = table(multinom_pred, test_pca$label)
multinom_acc = sum(diag(multinom_tab))/sum(multinom_tab)
multinom_tab
multinom_acc

# Random Forest
library(randomForest)
rdf_mod <- randomForest(label~., data=train, ntree=500)
rdf_pred <- predict(rdf_mod, test)

rdf_tab = table(rdf_pred, test$label)
rdf_acc = sum(diag(rdf_tab))/sum(rdf_tab)
rdf_tab
rdf_acc

```

```
drivers<-read.csv("./data/Drivers.csv", header = T, sep = ",")
fit.full<-glm(y~x1+x2+x3, data=drivers, family=binomial)
summary(fit.full)
```

系数估计与显著性检验

```
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.59761    0.89483   0.668   0.5042
## x1          -1.49608    0.70486  -2.123   0.0338 * 拟合里显著
## x2           -0.00160    0.01676  -0.095   0.9242
## x3           0.31587    0.70109   0.451   0.6523
```

Estimate: 系数的估计值

能写出回归方程: $\text{logit}(p) = 0.5976 - 1.4961 \times x_1 - 0.0016 \times x_2 + 0.3159 \times x_3$

Std. Error: 系数的标准误, 衡量估计的不确定性, 越小越精确

z value: 检验统计量 = Estimate / Std. Error, 用于检验该系数是否为0

Pr(>|z|): p值, < 0.05 → 该变量显著; 这里 x1 显著 (*), x2 x3 不显著

模型整体评估 (逻辑回归)

```
## Null deviance: 62.183 on 44 degrees of freedom
## Residual deviance: 57.026 on 41 degrees of freedom
## AIC: 65.026
```

Null deviance: 仅截距模型的偏差, 作为比较基准

Residual deviance: 当前模型的偏差, 越小越好, 表示拟合越好

Deviance 减少量: Null - Residual, 检验模型整体显著性

AIC: 赤池信息准则, 越小模型越好, 用于模型比较

整体显著性检验:

```
1 - pchisq(62.183 - 57.026, df = 44 - 41) # 卡方检验: 模型是否有意义, 结果>0.05?
模型整体不显著
```

```
# =====
# =====
```

线性回归 ANOVA

```
anova(model)
```

```
data(chickwts)
```

```
head(chickwts)
```

```
# 拟合线性回归模型 (等价于ANOVA)
```

```
model <- lm(weight ~ feed, data = chickwts)
```

```
anova(model)
```

线性回归的 ANOVA 表

```
## Analysis of Variance Table
##
## Response: weight
##              Df Sum Sq Mean Sq F value    Pr(>F)
## feed           5  231129    46226  15.365 5.936e-10 ***
## Residuals    65  195556     3009
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Sum Sq: 平方和 (组间/组内)

Mean Sq: 均方 = Sum Sq / Df

F value: F统计量 = 组间均方 / 组内均方

Pr(>F): p值, < 0.05 则表示至少有一组与其他组不同 (决定是否拒绝“所有组均值相等”的原假设)

结论

p = 5.94e-10 < 0.05 → 拒绝原假设 (几个mu全部相同) → 至少有一种饲料与其他饲料的增重效果不同

```
# =====  
=====
```

线性回归的 summary() 输出

```
## Residuals:  
##      Min       1Q   Median       3Q      Max  
## -5.6635 -1.4635  0.0254  1.3476  7.6478  
##  
## Coefficients:  
##              Estimate Std. Error t value Pr(>|t|)  
## (Intercept)  34.1167      1.3590   25.10  <2e-16 ***  
## dheight      0.4445      0.0213   20.87  <2e-16 ***  
##  
## Residual standard error: 2.053 on 1373 degrees of freedom  
## Multiple R-squared:  0.2408,    Adjusted R-squared:  0.2402  
## F-statistic: 435.5 on 1 and 1373 DF,  p-value: < 2.2e-16
```

Residuals: 中位数应该接近 0, 对称分布

残差的最小 1/4分位 中位数 3/4分位 最大

t value: 系数的 t 统计量, 绝对值 > 2 通常显著

Pr(>|t|): p 值, < 0.05 显著

Residual standard error: 残差标准差, 越小拟合越好

Multiple R-squared: 决定系数, 模型解释了多少变异 (0~1), 只适用于线性回归, 逻辑回归没有这个指标

Adjusted R-squared: 调整后 R², 惩罚变量个数, 用于模型比较

F-statistic & p-value: 模型整体显著性, p < 0.05 说明模型有意义

```
# =====  
=====
```

泊松回归

```
glm(.., family = poisson,)
```

```
library(alr4)  
data(AMSSurvey)  
head(AMSSurvey)
```

```
model <- glm(count ~ type + sex + citizen, family = poisson, data = AMSSurvey)  
summary(model)
```

拟合包含交互项的泊松回归 (更复杂的模型)

```
model_saturated <- glm(count ~ type * sex * citizen, family = poisson, data =  
  AMSSurvey)  
summary(model_saturated)
```

查看偏差和自由度

```
summary(model_full)$null.deviance      # Null deviance  
  仅包含截距的模型 (空模型) 的偏差 (基准)  
summary(model_full)$deviance           # Residual deviance 当前包含所有自变量的模型的偏差  
summary(model_full)$df.null            # Null df = n - 1 (样本量减1)  
summary(model_full)$df.residual        # Residual df = n - p - 1 (样本量减参数个数减1) !
```

拟合优度检验


```
p_value <- 1 - pchisq(deviance(model_full), df.residual(model_full))
p_value
```

判断: $p > 0.05$ 说明模型拟合良好, Residual Deviance 越小越好

Null deviance: 只有截距时, 模型无法解释的变异总量 (相当于线性回归中的 SST)

Residual deviance: 加入所有自变量后, 仍然无法解释的变异 (相当于线性回归中的 SSE)

两者的差值: 模型解释的变异量 (相当于线性回归中的 SSR)

检查模型整体显著性 (似然比检验), $p < 0.05$ 说明模型整体显著 (至少有一个自变量有效)

```
1 - pchisq(summary(model_full)$null.deviance - summary(model_full)$deviance,
           df = summary(model_full)$df.null - summary(model_full)$df.residual)
```

检查拟合优度 (模型是否合适), $p > 0.05$ 说明模型拟合良好 (残差偏差与自由度匹配)

```
1 - pchisq(summary(model_full)$deviance,
           df = summary(model_full)$df.residual)
```

泊松回归输出

```
## Null deviance: 521.4440 on 23 degrees of freedom
## Residual deviance: 1.9568 on 6 degrees of freedom
## AIC: 175.35
```

Null deviance: 521.444 on 23 df $\leftarrow$ 空模型: 偏差很大

Residual deviance: 1.957 on 6 df $\leftarrow$ 当前模型: 偏差大幅减小

残差偏差 $\approx$ 自由度 $\rightarrow$ 拟合良好

检验拟合优度

```
1 - pchisq(1.9568, df = 6) # 0.9236 > 0.05, 模型拟合良好
```

```
# =====
# =====
```

GAM (广义可加模型)

```
library(mgcv); library(alr4); data(pisa); head(pisa)
```

拟合广义可加模型 (GAM)

对 Income Edu Health 三个变量使用样条平滑

```
gam_model <- gam(Overall ~ s(Income) + s(Edu) + s(Health), data = pisa)
```

```
summary(gam_model)
```

```
## Approximate significance of smooth terms:
##      edf Ref.df      F  p-value
## s(Income) 7.593  8.415 8.826 1.29e-06 ***
## s(Edu)     6.204  7.177 3.308 0.00771 **
## s(Health) 1.000  1.000 2.736 0.10679
##
## R-sq.(adj) = 0.863 Deviance explained = 90.3%
```

s(Income) edf = 7.593 远大于1, 说明非线性关系很强 (曲线很弯曲)

(Income) p-value = 1.29e-06 ***, 极其显著, Income 对 Overall 有明显非线性影响

GAM 通过 s() 函数对变量进行非线性平滑拟合

edf: 有效自由度 (平滑程度), 越大说明曲线越弯曲 (非线性越强)

Ref.df: 参考自由度

F / p-value: 该平滑项是否显著 (判断该变量是否显著)

Deviance explained: 模型解释的偏差比例 (类似 R^2 , 和 $R\text{-sq.}(adj)$ 评估整体拟合优度)

```
# =====  
=====
```

Lasso 回归

```
library(glmnet); set.seed(123)
```

```
# 生成自变量 X (标准正态分布)
```

```
n <- 100
```

```
p <- 10
```

```
X <- matrix(rnorm(n * p), nrow = n, ncol = p)
```

```
# 设置真实的系数: 只有前3个变量有影响, 其余为0
```

```
beta <- c(1.2, -0.8, 0.5, rep(0, p - 3))
```

```
# 生成因变量 Y (加噪声)
```

```
Y <- X %*% beta + rnorm(n, sd = 0.5)
```

```
# 交叉验证选择最优 lambda
```

```
cv_lasso <- cv.glmnet(X, Y, alpha = 1) # alpha=1 表示 Lasso
```

```
# 查看最优 lambda
```

```
cv_lasso$lambda.min # 使交叉验证误差最小的 $\lambda$ ,  $\lambda$ 越大压缩越强  $\rightarrow$  更多系数被压缩为 0  $\rightarrow$   
模型更简单
```

```
cv_lasso$lambda.1se # 最稀疏的 $\lambda$  (误差在1个标准误内)
```

```
# 拟合完整 Lasso 路径
```

```
lasso_model <- glmnet(X, Y, alpha = 1)
```

```
# 提取 lambda.1se 对应的系数
```

```
coef(lasso_model, s = cv_lasso$lambda.1se)
```

Lasso 回归系数输出:

```
coef(obj, s = obj.cv$lambda.1se)
```

```
## 4 x 1 sparse Matrix
```

```
## (Intercept) 0.089
```

```
## X1 0.923 # 被选中
```

```
## X2 0.846 # 被选中
```

```
## X3 . # 被压缩为 0, 剔除
```

```
## > cv_lasso$lambda.min
```

```
## [1] 0.05423167 # 较小  $\rightarrow$  模型较复杂, 预测误差最小
```

```
## > cv_lasso$lambda.1se
```

```
## [1] 0.1523456 # 较大  $\rightarrow$  模型更简单, 但预测误差在1个标准误内
```

```
## > coef(lasso_model, s = cv_lasso$lambda.1se)
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
```

```
s1
```

```
## (Intercept) 0.023456789
```

```
## V1 1.156789012
```

```
## V2 -0.745678901
```

```
## V3 0.423456789
```

```
## V4 .
```

```
## V5 .
```

```
## V6 .
```

```
## V7 .
```

```
## V8 .
```

```
## V9 .
```

```
## V10 .
```

```
# 二分类问题
```

```

Y_binary <- ifelse(Y > median(Y), 1, 0)
# 交叉验证 (用 family="binomial")
cv_lasso_logit <- cv.glmnet(X, Y_binary, alpha = 1, family = "binomial")
# 提取系数
coef_logit <- coef(cv_lasso_logit, s = "lambda.1se")
coef_logit[coef_logit != 0, ]

# =====
# =====

```

聚类/降维

```

data(USArrests); head(USArrests)
# 数据标准化 (PCA 前必须做, 因为变量单位不同)
df_scaled <- scale(USArrests)
# PCA
pca_result <- prcomp(df_scaled, center = TRUE, scale. = TRUE)
# 方差解释率表
summary(pca_result)

```

```

## Importance of components:
##
##          Comp.1      Comp.2      Comp.3
## Standard deviation  5.133350  4.144921  3.655005
## Proportion of Variance 0.055593  0.036246  0.028184
## Cumulative Proportion 0.055593  0.091839  0.120022

```

Standard deviation: 特征值的平方根

Proportion of Variance: 单个主成分解释的方差比例

Cumulative Proportion: 累积解释比例 (选择保留多少PC的依据)

```

# =====
# =====

```

方差齐性检验 - 检验线性回归模型的【残差】是否满足方差齐性假设

```
ncvTest(model)
```

先拟合一个线性回归模型

```

model <- lm(y ~ x1 + x2, data = mydata)
# 然后用 ncvTest() 检验方差齐性, p<0.05 表示存在异方差
library(car)
ncvTest(model)

```

```

# Non-constant Variance Score Test
# Variance formula: ~ fitted.values
# Chisquare = 12.34567    Df = 1    p = 0.00044

```

$p < 0.05$ → 拒绝原假设 (方差齐) → 存在异方差

$p \geq 0.05$ → 不拒绝原假设 → 方差齐性成立

```
plot(model, which = 1) # 残差vs拟合值图, 点随机分布对应 $p \geq 0.05$ , 方差齐性
```

```

#####
s1 <- lm(Y ~ ., sniffer)
ncvTest(s1) # 默认基于拟合值
ncvTest(s1, ~ TankTemp + GasTemp) # 指定变量

```

基于拟合值

```

## Non-constant Variance Score Test
## Chisquare = 4.802652, Df = 1, p = 0.028416

```

```
# 基于指定变量
## Chisquare = 9.83621, Df = 2, p = 0.007313

p < 0.05 → 存在异方差，需要加权最小二乘法或变换因变量
```

```
# =====
# =====
```

```
残差诊断图 (Residual Plots)
obj <- lm(dheight ~ mheight, data = Heights)
residualPlots(obj)

plot(lm_model)
```

依次检查

残差 vs 拟合值：是否有非线性趋势或异方差
 Q-Q 图：残差是否正态（点应接近直线）
 Scale-Location：方差齐性（水平线最好）
 残差 vs 杠杆值：是否有强影响点

```
# =====
```

```
异常值检测 (Outlier Test)
obj <- lm(lpres ~ bp, data = Forbes)
outlierTest(obj)
```

```
##          rstudent  unadjusted p-value  Bonferroni p
##  12  12.40691          6.0892e-09    1.0352e-07
```

第12个观测点是异常值（学生化残差极大，Bonferroni校正后 $p < 0.05$ ）
 处理方式：删除后重新拟合（Forbes1 = Forbes[-12,]）

异常值/强影响点

```
outlierTest(model)          # 输出异常点的编号和p值
influenceIndexPlot(model) # 强影响点可视化
```

```
##          Test stat Pr(>|Test stat|)
## mheight      0.1458      0.8841
## Tukey test    0.1458      0.8841
```

Test stat: 检验统计量

Pr(>|Test stat|): p值

p > 0.05 → 没有证据表明存在非线性或异方差（好）

p < 0.05 → 存在非线性关系，考虑对变量做变换

如果 ## Tukey test 5.4198 5.966e-08 ***
 p < 0.05 → 拒绝线性假设，说明模型缺少非线性项或交互项

```
强影响点诊断 (Influence Index Plot)
influenceIndexPlot(obj)
```

输出四张图

Cook's distance: 衡量每个点对系数估计的影响，> 1 → 强影响点
 hat values: 杠杆值（高杠杆点）> $2p/n$ → 高杠杆点
 studentized residuals: 学生化残差（异常值）> 2 → 可能的异常值
 Cook's dist vs hat: 综合图

```
# =====
# =====
```

分类模型混淆矩阵

```
table(predicted, actual)
##           actual
## predicted Pass Fail
##      Pass  420   23
##      Fail   20    8
```

```
accuracy <- (420 + 8) / (420 + 23 + 20 + 8) # 0.908
```

```
# =====
# =====
```

Box-Cox变换 (Power Transform + Inverse Response Plot)

场景：找出自变量需要做什么变换

输入1: powerTransform (对多个自变量)

```
Highway$sigs1 <- with(Highway, (sigs * len + 1)/len)
pT <- powerTransform(cbind(len, adt, trks, shld, sigs1) ~ 1, Highway)
summary(pT)
```

输出1:

```
##           Est Power Rounded Pwr Wald Lwr Bnd Wald Up Bnd
## len           0.1437           0      -0.2732           0.5607
## adt           0.0509           0      -0.1854           0.2872
## trks          -0.7028           0      -1.9134           0.5078
## shld           1.3456           1       0.6341           2.0570
## sigs1         -0.2408           0      -0.5341           0.0525
```

Rounded Pwr 列：建议的变换

0 → 对数变换 $\log(x)$; 0.5 → 平方根变换 $\sqrt{x}$

1 → 不变换; -1 → 倒数变换 $1/x$

输入2: inverseResponsePlot (对因变量)

```
inverseResponsePlot(m2)
```

输出2:

```
##           lambda      RSS
## 1  0.1197427 32.32520
## 2 -1.0000000 35.70143
## 3  0.0000000 32.36532
## 4  1.0000000 34.24662
```

第1行是最优lambda (≈ 0.12)，接近0 → 建议对因变量做对数变换

对比RSS: lambda=0时RSS=32.365, lambda=1时RSS=34.247, log变换更好

散点图与变换 (scatterplot + invTranPlot)

```
scatterplot(BrainWt ~ BodyWt, brains, id = TRUE) # 找出异常点
scatterplot(log(BrainWt) ~ log(BodyWt), brains) # 对数变换后线性关系更好
```

```
plot(Height ~ Dbh, ufcwc)
invTranPlot(ufcwc$Dbh, ufcwc$Height) # 找最优变换
```

```
##           lambda      RSS
## 1  0.04787804 152122.8
## 2 -1.00000000 197352.2
```

```
## 3 0.00000000 152232.3
## 4 1.00000000 193739.7
```

最优 $\lambda \approx 0.05$, 接近0 $\rightarrow$ 建议 $\log(\text{Dbh})$

QQ图 (正态性检验)

```
m2 <- lm(fertility ~ log(ppgdp), UN11)
qqPlot(m2, id = TRUE)
```

```
## Equatorial Guinea      Zambia
##                      58      198
```

图中标注的点是偏离正态最严重的观测,可用于识别需要进一步检查的异常值

```
# =====
# =====
```

完整流程

拟合初始模型

```
model <- lm(y ~ x1 + x2, data = df)
```

残差诊断

```
residualPlots(model)      # 检查非线性和交互项需求
```

异常值检测

```
outlierTest(model)        # 找出异常点
```

```
influenceIndexPlot(model) # 找出强影响点
```

方差齐性

```
ncvTest(model)            # 检查异方差
```

如果需要变换

```
inverseResponsePlot(model)      # 找因变量变换
```

```
powerTransform(~ x1 + x2, data = df) # 找自变量变换
```

改进模型

```
model2 <- lm(log(y) ~ log(x1) + sqrt(x2), data = df)
```

验证改进

```
residualPlots(model2)      # 应得到改善
```

```
# =====
# =====
```

输出速查

输出	为什么关键	$p < 0.05$ 意味着
residualPlots 的 $\text{Pr}(> \text{Test stat})$	每个变量的线性假设	该变量需要非线性项
residualPlots 的 Tukey test	整体线性假设	模型缺少交互项或非线性项
ncvTest	方差齐性	存在异方差
outlierTest 的 Bonferroni p	异常值	该点是异常值
influenceIndexPlot	Cook's distance	> 1 的点是强影响点
powerTransform 的 Rounded Pwr	自变量变换	0=log, 0.5=sqrt, 1=不变
inverseResponsePlot	因变量变换	λ 接近0时用log 接近1时不变
qqPlot	正态性	标出的点偏离正态

```

# =====
# 1. 缺失值处理
# =====

# 填补缺失值：用该列均值填充
for(i in 1:ncol(df)) {
  if(any(is.na(df[, i]))) {
    col_mean <- mean(df[, i], na.rm = TRUE)
    df[is.na(df[, i]), i] <- col_mean
  }
}

# 检查缺失值数量
sum(is.na(df))

# =====
# 2. 剔除常数列（方差为0的列）
# =====

constant_vars <- sapply(df, function(x) length(unique(x)) == 1)
df_filtered <- df[, !constant_vars]

# =====
# 3. 数据标准化（Z-score）
# =====

df_scaled <- scale(df)

# =====
# 4. 分离数值列和非数值列
# =====

numeric_cols <- sapply(df, is.numeric)
df_numeric <- df[, numeric_cols, drop = FALSE]
df_non_numeric <- df[, !numeric_cols, drop = FALSE]

# =====
# 5. 常见分布的概率计算
# =====

# 正态分布
pnorm(85, mean=80, sd=5)           # P(X ≤ 85)
qnorm(0.975, mean=0, sd=1)         # 分位数：1.96
dnorm(x, mean=0, sd=1)             # 密度函数值

# 二项分布
dbinom(500, size=5000, prob=0.1)   # P(X = 500)
pbinom(199, size=1000, prob=0.2)   # P(X ≤ 199)

# 泊松分布
dpois(5, lambda=10)                # P(X = 5)
ppois(60, lambda=50)               # P(X ≤ 60)

# 指数分布
dexp(x, rate=0.5)                   # 密度函数
rexp(100, rate=0.5)                # 生成随机数

# =====
# 6. t分布、F分布、卡方分布的分位数
# =====

```

```

qt(0.975, df=20)                # t分布分位数
qf(0.975, df1=10, df2=20)       # F分布分位数
qchisq(0.95, df=10)            # 卡方分布分位数

# =====
# 7. 置信区间计算
# =====

# 正态分布均值的置信区间 (σ已知)
c(mean(X) - 1.96 * sigma/sqrt(n), mean(X) + 1.96 * sigma/sqrt(n))

# t分布均值的置信区间 (σ未知)
mean_x <- mean(X)
se <- sd(X) / sqrt(n)
t_crit <- qt(0.975, df=n-1)
c(mean_x - t_crit * se, mean_x + t_crit * se)

# 一键计算
t.test(X)$conf.int

# 方差的置信区间 (卡方分布)
df <- n-1
chi_lower <- qchisq(0.025, df)
chi_upper <- qchisq(0.975, df)
c(df * var(X) / chi_upper, df * var(X) / chi_lower)

# =====
# 8. 单样本t检验
# =====

t.test(X, mu=0)                # 检验均值是否为0

# =====
# 9. 两样本t检验
# =====

# 等方差假设
t.test(X, Y, var.equal=TRUE)

# 方差不齐时 (Welch检验, 默认)
t.test(X, Y)

# =====
# 10. 方差齐性检验 (F检验)
# =====

var.test(X1, X2)

# =====
# 11. 卡方检验 (拟合优度/独立性)
# =====

# 拟合优度检验
chisq.test(observed_counts)

# 列联表独立性检验
chisq.test(contingency_table, correct=FALSE)

```



```

# Fisher精确检验 (小样本)
fisher.test(contingency_table)

# =====
# 12. 简单线性回归
# =====

model <- lm(y ~ x, data=df)
summary(model)           # 查看系数、R2、p值
confint(model)           # 系数的置信区间

# =====
# 13. 多元线性回归
# =====

model <- lm(y ~ x1 + x2 + x3, data=df)
summary(model)

# 预测
predict(model, newdata=new_df)

# =====
# 14. 模型诊断
# =====

# 残差图
plot(model$residuals ~ df$x)
# 异常值检测
outlierTest(model)
# 多重共线性检验 (VIF)
vif(model)

# =====
# 15. 单因素ANOVA
# =====

aov_fit <- aov(y ~ group, data=df)
summary(aov_fit)

# 事后检验 (Tukey HSD)
TukeyHSD(aov_fit)

# =====
# 16. 虚拟变量回归 (等价于ANOVA)
# =====

# R自动将第一个因子水平设为基准组
lm(weight ~ feed, data=chickwts)

# 查看基准组 (第一个水平)
levels(df$group)

# 线性回归
lm(weight ~ feed, data = chickwts)

# 方差分析
aov(weight ~ feed, data = chickwts)

```

这两个模型在数学上是完全等价的：

lm() 输出每个组与基准组的差值 (回归系数)

aov() 直接输出 F 检验, 判断各组均值是否相等

```
summary(aov(weight ~ feed, data = chickwts))
```

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
feed	5	231129	46226	15.37	5.94e-10 ***
Residuals	65	195556	3009		

lm() 的 F 检验结果完全一样 (从 summary(lm(...)) 最后一行可以找到)

改变基准组: relevel() 函数

```
# 将 soybean 设为基准组
```

```
chickwts$feed <- relevel(chickwts$feed, ref = "soybean")
```

```
lm(weight ~ feed, data = chickwts)
```

```
# =====  
# 17. 逻辑回归 (二分类)  
# =====
```

```
glm(y ~ x1 + x2, family=binomial, data=df)
```

```
# 预测概率
```

```
predict(model, newdata, type="response")
```

```
# =====  
# 18. 泊松回归 (计数数据)  
# =====
```

```
glm(count ~ x1 + x2, family=poisson, data=df)
```

```
# =====  
# 19. 多项逻辑回归 (多分类)  
# =====
```

```
library(nnet)
```

```
multinom(outc ~ hia + gravi, data=Ectopic)
```

```
# =====  
# 20. 主成分分析 (PCA)  
# =====
```

```
pca <- prcomp(df, center=TRUE, scale.=TRUE)
```

```
# 查看方差解释率
```

```
summary(pca)
```

```
# 累积方差图
```

```
cum_var <- cumsum(pca$sdev^2 / sum(pca$sdev^2))
```

```
# 提取主成分得分
```

```
pca_scores <- pca$x[, 1:2]
```

```
# =====  
# 21. t-SNE (非线性降维)  
# =====
```

```
library(Rtsne)
```

```
set.seed(42)
```

```

tsne_out <- Rtsne(X, dims=2, perplexity=30, verbose=TRUE)
tsne_df <- data.frame(tSNE1=tsne_out$Y[,1], tSNE2=tsne_out$Y[,2])

# =====
# 22. K-means聚类
# =====

# 选择K值 (肘部法则)
fviz_nbclust(df, kmeans, method="wss")

km <- kmeans(df, centers=4, nstart=24)

# =====
# 23. 层次聚类
# =====

dist_mat <- dist(df, method="euclidean")
hc <- hclust(dist_mat, method="ward.D2")
plot(hc)
clusters <- cutree(hc, k=3)

# =====
# 24. 谱聚类
# =====

library(kernlab)
sc <- specc(as.matrix(df), centers=2)

# =====
# 25. 高斯混合模型 (GMM)
# =====

library(mclust)
em <- Mclust(df, G=3)
summary(em)
plot(em, what="classification")

# =====
# 26. Lasso回归
# =====

library(glmnet)

# 交叉验证选择lambda
cv_obj <- cv.glmnet(X, Y, alpha=1) # alpha=1是Lasso

# 最优lambda
cv_obj$lambda.min # 最小MSE
cv_obj$lambda.1se # 1se规则 (更稀疏)

# 提取系数
coef(cv_obj, s="lambda.1se")

# 画解路径
plot(glmnet(X, Y), label=TRUE)

# =====
# 27. 逐步回归 (基于AIC)
# =====

```

```

# 向前选择
step(m0, scope=~ x1 + x2 + x3, direction="forward")

# 向后剔除
step(m1, direction="backward")

# =====
# 28. 线性判别分析 (LDA)
# =====

library(MASS)
lda_model <- lda(Label ~ ., data=train)
predictions <- predict(lda_model, test)

# =====
# 29. 朴素贝叶斯
# =====

library(e1071)
nb_model <- naiveBayes(Label ~ ., data=train)
pred <- predict(nb_model, test)

# =====
# 30. 支持向量机 (SVM)
# =====

library(kernlab)
svm_model <- ksvm(Label ~ ., data=train,
                  type="C-bsvc", kernel="rbfdot")

# =====
# 31. 决策树 (CART)
# =====

library(rpart)
tree_model <- rpart(Label ~ ., data=train, method='class')

# 剪枝
printcp(tree_model)
pruned_tree <- prune(tree_model, cp=0.01)

# =====
# 32. 随机森林
# =====

library(randomForest)
rf_model <- randomForest(Label ~ ., data=train, ntree=500)

# 变量重要性
varImpPlot(rf_model)

# =====
# 33. 模型评估：混淆矩阵
# =====

conf_mat <- table(predicted, actual)
accuracy <- sum(diag(conf_mat)) / sum(conf_mat)

# =====
# 34. SMOTE过采样
# =====

```

```

library(smotefamily)
smote_res <- SMOTE(X_train, y_train, K=5, dup_size=2)
train_balanced <- smote_res$data

# =====
# 35. GAM (样条平滑)
# =====

library(mgcv)

# 线性模型
mod_lm <- gam(Overall ~ Income, data=pisa)

# 带平滑项的GAM
mod_gam <- gam(Overall ~ s(Income) + s(Edu) + s(Health), data=pisa)

# 查看平滑项显著性
summary(mod_gam)
plot(mod_gam) # 可视化平滑曲线

# 模型比较
anova(mod_lm, mod_gam, test="Chisq")

# =====
# 36. 条件逻辑回归 (clogit)
# =====

library(survival)
clogit_model <- clogit(sAKI ~ BMI + Cr + Lactate + strata(subclass),
                      data=mydata, method="exact")
summary(clogit_model)

# =====
# 37. Gibbs采样示例：变点检测
# =====

gibbs_sampler_changepoint <- function(y, n_iter, a=1, b=1, c=1, d=1) {
  n <- length(y)
  # 初始化
  k <- sample(2:(n-1), 1)
  lambda <- rgamma(1, shape=a, rate=b)
  phi <- rgamma(1, shape=c, rate=d)

  for (i in 1:n_iter) {
    # 更新 lambda (前半段)
    a_post <- a + sum(y[1:k])
    b_post <- b + k
    lambda <- rgamma(1, shape=a_post, rate=b_post)

    # 更新 phi (后半段)
    c_post <- c + sum(y[(k+1):n])
    d_post <- d + (n - k)
    phi <- rgamma(1, shape=c_post, rate=d_post)

    # 更新 k (变点位置)
    # ... (计算后验概率并采样)
  }
  return(list(lambda=lambda, phi=phi, k=k))
}

```

```

}

# =====
# 38. 常用公式
# =====

# 标准误
se <- sd(x) / sqrt(n)

# 合并方差 (两样本t检验)
pooled_var <- ((n1-1)*var1 + (n2-1)*var2) / (n1 + n2 - 2)

# 决定系数 R2
R_squared <- 1 - sum(residuals^2) / sum((y - mean(y))^2)

# 调整R2
adj_R_squared <- 1 - (1-R_squared) * (n-1)/(n-p-1)

# AIC
AIC <- -2 * logLik(model) + 2 * p

# BIC
BIC <- -2 * logLik(model) + p * log(n)

# McFadden伪R2 (逻辑回归)
pseudo_R2 <- 1 - (model$deviance / model$null.deviance)

```